



Functions

We should now understand the different branching and control instructions used to control the program's execution flow. We should also have a proper grasp of procedures and calls and how to utilize them for branching. So, let's now focus on calling functions.

Functions Calling Convention

Functions are a form of `procedures`. However, functions tend to be more complex and should be expected to use the stack and all registers fully. So, we can't simply call a function as we did with procedures. Instead, functions have a `Calling Convention` to properly set up before being called.

There are four main things we need to consider before calling a function:

1. `Save Registers` on the stack (`Caller Saved`)
2. Pass `Function Arguments` (like syscalls)
3. `Fix Stack Alignment`
4. Get Function's `Return Value` (in `rax`)

This is relatively similar to calling a syscall, and the only difference with syscalls is that we have to store the syscall number in `rax`, while we can call functions directly with `call function`. Furthermore, with syscall we don't have to worry about `Stack Alignment`.

Writing Functions

All of the above points are from a `caller` point of view, as we call a function. When it comes to writing a function, there are different points to consider, which are:

1. Saving `Callee Saved` registers (`rbx` and `rbp`)
2. Get arguments from registers
3. Align the Stack
4. Return value in `rax`

As we can see, these points are relatively similar to the `caller` points. The `caller` is setting up things, and then the `callee` (i.e., receiver) should retrieve those things and use them. These points are usually made at the beginning, and the end of the function and are called a function's `prologue` and `epilogue`. They allow functions to be called without worrying about the current state of the stack or the registers.

In this module, we will only be calling other functions, so we only have to focus on setting up a function call and won't go into writing functions.

Using External Functions

We want to print the current Fibonacci number at each iteration of the `loopFib` loop. Previously, we could not use a `write` syscall since it only accepts `ASCII` characters. We would have had to convert our Fibonacci number to `ASCII`, which is a bit complicated.

Luckily, there are external functions we can use to print the current number without having to convert it. The `libc` library of functions used for `C` programs provides many functionalities that we can utilize without rewriting everything from scratch. The `printf` function in `libc` accepts the printing format, so we can pass it the current Fibonacci number and tell it to print it as an integer, and it'll do the conversion automatically. Before we can use a function from `libc`, we have to import it first and then specify the `libc` library for dynamic linking when linking our code with `ld`.

Importing libc Functions

First, to import an external `libc` function, we can use the `extern` instruction at the beginning of our code, as follows:

```
global _start
extern printf
```

nasm

Once this is done, we should be able to call the `printf` function. So, we can proceed with the `Functions Calling Convention` we discussed earlier.

Saving Registers

Let's define a new procedure, `printFib`, to hold our function call instructions. The very first step is to save to the stack any registers we are using, which are `rax` and `rbx`, as follows:

nasm

```
printfib:
    push rax          ; push registers to stack
    push rbx
    ; function call
    pop rbx           ; restore registers from stack
    pop rax
    ret
```

So, we can proceed with the second point, and pass the required arguments to `printf`.

Function Arguments

We have already discussed how to pass function arguments in the syscall section. The same process applies to function arguments.

First, we need to find out what arguments are accepted by the `printf` function by using `man -s 3` for `library functions manual` (as we can see in `man man`):

```
shellsession
kimvutrinh@htb[/htb]$ man -s 3 printf

...SNIP...
    int printf(const char *format, ...);
```

As we can see, the function takes a pointer to the print format (shown with a `*`), and then the string(s) to be printed.

First, we can create a variable that contains the output format to pass it as the first argument. The `printf` man page also details various print formats. We want to print an integer, so we can use the `%d` format, as follows:

```
nasm

global _start
extern printf

section .data
    message db "Fibonacci Sequence:", 0x0a
    outFormat db "%d", 0x0a, 0x00
```

Note: We ended the format with a null character `0x00`, as this is the string terminator in `printf`, and we must terminate any string with it.

This can be our first argument, and `rbx` as our second argument, which `printf` will place as `%d`. So, let's move both arguments to their respective registers, as follows:

```
nasm

printFib:
    push rax                ; push registers to stack
    push rbx
    mov rdi, outFormat      ; set 1st argument (Print Format)
    mov rsi, rbx            ; set 2nd argument (Fib Number)
    pop rbx                ; restore registers from stack
    pop rax
    ret
```

Stack Alignment

Whenever we want to make a `call` to a function, we must ensure that the `Top Stack Pointer (rsp)` is aligned by the `16-byte boundary` from the `_start` function stack.

This means that we have to push at least 16-bytes (or a multiple of 16-bytes) to the stack before making a call to ensure functions have enough stack space to execute correctly. This requirement is mainly there for processor performance efficiency. Some functions (like in `libc`) are programmed to crash if this boundary is not fixed to ensure performance efficiency. If we assemble our code and break right after the second `push`, this is what we will see:

```
stack — 0x00007fffffffe3a0 | +0x0000: 0x0000000000000001 ←
$rsp 0x00007fffffffe3a8 | +0x0008: 0x0000000000000000
0x00007fffffffe3b0 | +0x0010: 0x00000000004010ad → <loopFib+5> add rax,
rbx 0x00007fffffffe3b8 | +0x0018: 0x0000000000401044 → <_start+20> call 0x4010bd
<Exit> 0x00007fffffffe3c0 | +0x0020: 0x0000000000000001 ←
$r13 —
code:x86:64 —
    0x401090 <initFib+9>      ret
    0x401091 <printFib+0>    push    rax
    0x401092 <printFib+1>    push    rbx
→ 0x40100e <printFib+2>    movabs  rdi, 0x403039
```

We see that we have four 8-bytes pushed to the stack, making a total boundary of 32-bytes. This is due to two things:

1. Each procedure `call` adds an 8-byte address to the stack, which is then removed with `ret`
2. Each `push` adds 8-bytes to the stack as well

So, we are inside `printFib` and inside `loopFib`, and have pushed `rax` and `rbx`, for a total of a 32-byte boundary. Since the boundary is a multiple of 16, `our stack is already aligned, and we don't have to fix anything.`

If we were in a case where we wanted to bring the boundary up to 16, we can subtract bytes from `rsp`, as follows:

```
sub rsp, 16
call function
add rsp, 16
```

nasm

This way, we are adding an extra 16-bytes to the top of the stack and then removing them after the call. If we had 8 bytes pushed, we can bring the boundary up to 16 by subtracting 8 from `rsp`.

This may be a bit confusing, but the critical thing to remember is that `we should have 16-bytes (or a multiple of 16) on top of the stack before making a call.` We can count the number of (un `pop` ed) `push` instructions and (un `ret` urned) `call` instructions, and we will get how many 8-bytes have been pushed to the stack.

Function Call

Finally, we can issue `call printf`, and it should print the current Fibonacci number in the format we specified, as follows:

```
printFib:
    push rax                ; push registers to stack
```

nasm

```

push rbx
mov rdi, outFormat ; set 1st argument (Print Format)
mov rsi, rbx       ; set 2nd argument (Fib Number)
call printf        ; printf(outFormat, rbx)
pop rbx            ; restore registers from stack
pop rax
ret

```

Now we should have our `printFib` procedure ready. So, we can add it to the beginning of `loopFib`, such that it prints the current Fibonacci number at the beginning of each loop:

```

loopFib:
    call printFib ; print current Fib number
    add rax, rbx  ; get the next number
    xchg rax, rbx ; swap values
    cmp rbx, 10   ; do rbx - 10
    js loopFib    ; jump if result is <0
    ret

```

Our final `fib.s` code should be as follows:

```

global _start
extern printf

section .data
    message db "Fibonacci Sequence:", 0x0a
    outFormat db "%d", 0x0a, 0x00

```

```

section .text
_start:
    call printMessage    ; print intro message
    call initFib         ; set initial Fib values
    call loopFib         ; calculate Fib numbers
    call Exit            ; Exit the program

printMessage:
    mov rax, 1           ; rax: syscall number 1
    mov rdi, 1           ; rdi: fd 1 for stdout
    mov rsi, message     ; rsi: pointer to message
    mov rdx, 20          ; rdx: print length of 20 bytes
    syscall              ; call write syscall to the intro message
    ret

initFib:
    xor rax, rax         ; initialize rax to 0
    xor rbx, rbx         ; initialize rbx to 0
    inc rbx              ; increment rbx to 1
    ret

printFib:
    push rax             ; push registers to stack
    push rbx
    mov rdi, outFormat   ; set 1st argument (Print Format)
    mov rsi, rbx         ; set 2nd argument (Fib Number)
    call printf          ; printf(outFormat, rbx)
    pop rbx              ; restore registers from stack
    pop rax
    ret

loopFib:
    call printFib        ; print current Fib number

```

```
add rax, rbx      ; get the next number
xchg rax, rbx     ; swap values
cmp rbx, 10       ; do rbx - 10
js loopFib        ; jump if result is <0
ret
```

Exit:

```
mov rax, 60
mov rdi, 0
syscall
```

Dynamic Linker

We can now assemble our code with `nasm`. When we link our code with `ld`, we should tell it to do dynamic linking with the `libc` library. Otherwise, it would not know how to fetch the imported `printf` function. We can do so with the `-lc --dynamic-linker /lib64/ld-linux-x86-64.so.2` flags, as follows:

```
shellsession
kimvutrinh@htb[/htb]$ nasm -f elf64 fib.s && ld fib.o -o fib -lc --dynamic-linker /lib64/ld-linux-x86-64.so.2 && ./fib
```

```
1
1
2
3
5
8
```

As we can see, `printf` made it very easy to print our Fibonacci number without worrying about converting it to the proper format, like we had to with the `write` syscall. Next, we need to go through another example of using external `libc` functions to understand how to call external functions properly.

Connect to HTB



Pwnbox

Your own web-based Parrot Linux instance to play our labs.



Switching Pwnbox location will terminate the spawned Pwnbox.

Pwnbox Location

UK



Start Pwnbox

∞ spawns left

• Offline



Enable step-by-step solutions ⓘ

PRO

⚠ Answer questions in English to ensure an accurate response.

Question 1

+2

+40



Try to fix the Stack Alignment in "print", so it does not crash, and prints "Its Aligned!". How much boundary was needed to be added? "write a number"

Write your answer

Submit



Hint



Download File

← Previous

Section 19 / 24

 +20

Next

Mark Complete & Next